

Predlog projekta iz predmeta Sistemi bazirani na znanju

Uroš Radukić SV54/2022

Opis problema

Motivacija

Trenutno razvijam video igru za [Nordeus Challenge](https://nordeus.com/nordeus-challenge/full-stack/) (<https://nordeus.com/nordeus-challenge/full-stack/>). U pitanju je strateška igra po potezima (turn-based RPG), gde igrač i neprijatelj naizmenično biraju kako će iskoristiti svoj potez, dok ne ostane samo jedan od njih živ. Jedna od stavki u izazovu koja je poželjna je da neprijateljski AI ne bira nasumično kako će iskoristiti svoj potez, već da na smislen način odabere koji mu je najbolji potez. Veštačka inteligencija neprijatelja u video igrama je jedan od ključnih faktora koji utiču na kvalitet iskustva igrača. To je posebno izraženo u žanru strateških igara po potezima, ponašanje neprijatelja direktno utiče na to koliko je igra zanimljiva, i koliki izazov predstavlja. Tu sam uvideo idealnu priliku da iskoristim Drools i znanje stečeno na predmetu Sistemi bazirani na znanju i uz svoje ekspertsko znanje iz oblasti strateških video igara dizajniram AI za neprijatelje.

Pregled problema

Postojeća rešenja i njihova ograničenja

Konačni automati (FSM) dugo su bili standard u video igrama. Svako stanje definiše ponašanje, a tranzicije reaguju na događaje. Prednost je jednostavnost implementacije, ali mana je što nema nikakvu fleksibilnost, jer neprijatelj ima fiksni broj stanja u kojima može biti.

Stabla ponašanja (Behavior Trees), popularizovana u video igrama kao što su Halo i The Sims, nude hijerarhijsku kompoziciju akcija i uslova. Fleksibilnija su od FSM-a, ali i dalje ručno projektovana i neintuitivna za zaključivanje. Takođe, ne postoji prirodan mehanizam za detekciju obrazaca kroz vreme.

Utility AI, koji se koristi u The Sims 4, dodeljuje numeričke vrednosti mogućim akcijama na osnovu konteksta. Mada fleksibilan, logika donošenja odluka postaje teška za praćenje, a i implementiranje. Sistem postaje nepredvidiv pri većem broju faktora i teško ga je ispratiti.

Mašinsko učenje i neuronske mreže nude adaptivnost, ali zahtevaju skupe trening podatke, neprozirne su i nedeterminističke, što nije prikladno za igrenu logiku gde je konzistentnost ponašanja neprijatelja od suštinskog značaja.

Naše rešenje i prednosti

Ideja projekta je da se AI neprijatelja implementira korišćenjem **Drools rule engine-a**. Ovaj pristup donosi prednosti:

- **EksPLICITNA pravila** čitljiva kao poslovne politike. Ovo je posebno značajno u razvoju video igara zbog toga što omogućava dizajnerima da ručno unose pravila sa minimalnim znanjem programiranja. To olakšava ceo proces iterativnog razvoja jer se gubi potreba za prisustvom programera pri svakom testiranju novih parametara.
- **CEP (Complex Event Processing)** za detekciju obrazaca tokom borbenih događaja korišćenjem klizajućih prozora (sliding windows). Umesto da posmatramo konkretno vremensku odredbu, posmatračemo prednoinih N poteza i gledati da donesemo neki zaključak. Jedna od dobrih osobina neprijateljskog AI bila bi da se i adaptira na konkretno, specifično ponašanje igrača, na primer ukoliko je igrač u poslednjih nekoliko poteza samo koristio fizičke napade, možemo zaključiti da povećavanje odbrane ima veći značaj nego inače.
- **Backward chaining** za ciljno zaključivanje, poput toga "Kojim potezom/kombinacijom poteza bi mogao da ubije igrača". Na osnovu konkretnog cilja gleda kako može da ga ispuni.
- **Višeslojnu arhitekturu** (L1 percepcija → L2 taktika → L3 akcija) koja razdvaja odgovornosti i nudi fleksibilnost pri biranju konkretnog poteza
- **Arhetipovi** korišćenjem Template-a. Kako bi se bolje izrazile razlike u neprijateljima i njihove karakterne osobine (a bez dodatne komplikacije koja bi nasledila u ostalim pristupima), možemo koristiti template za podesavanje različitih pragova za činjenice. Na primer, za činjenicu `HEALTH_CRITICAL`, drugačiji prag bismo imali za goblina koji je plasiljiv (recimo 40%) i za viteza koji je neustrasiv (10%).

Metodologija rada

Ulazi u sistem (Input)

Za svaki potez neprijatelja sistem prima sledeće činjenice koje se upisuju u Drools radnu memoriju:

Činjenica	Sadržaj
EnemyFact	Arhitip, HP (trenutni i maksimalni), stamina, mana, statistike (napad, odbrana, magija)
PlayerFact	HP (trenutni i maksimalni), statistike (napad, odbrana, magija)
MoveOption	Lista poteza koje neprijatelj može da priušti: za svaki stoji kategorija, cena, projektivna vrednost štete
ActiveStatusEffect	Aktivni statusni efekti na igraču i neprijatelju (buff/debuff)
CombatEventFact	Istorija borbenih događaja unesena u CEP event stream (<code>combat-stream</code>)

Izlazi iz sistema (Output)

Sistem proizvodi jednu odluku po pozivu:

- **EnemyDecision**: identifikator poteza koji neprijatelj treba da odigra, sa prioritetom i tekstualnim obrazloženjem

Iz skupa svih generisanih odluka bira se ona sa najvišim prioritetom i vraća `CombatService-u` koji je izvršava.

Baza znanja

Popunjavanje baze znanja: Pravila su statički definisana u DRL fajlovima i učitavaju se pri pokretanju aplikacije. Konfiguracija likova, poteza i predmeta čita se iz JSON fajlova (`characters.json`, `moves.json`, `items.json`) — dodavanje novog neprijatelja ili poteza ne zahteva izmenu Java koda, već samo dodavanje u JSON fajl.

Interakcije na osnovu znanja odvijaju se u kaskadi po salience vrednostima:

- L1 pravila (salience 100) izvlače apstraktne **činjenice** iz konkretnih vrednosti
- CEP pravila izvlače informacije o teku bitke i obrascu ponašanja igrača
- Backward chaining query `canKillPlayer` se aktivira prvi i traži potez kojim bi mogao ubiti igrača, jer ako takav potez postoji dalje pretrage nemaju smisla. Ako pronađe takav potez, odmah ga vraća, a ako ne uspe, rezonovanje se nastavlja normalno
- L2 pravila (salience 35–50) biraju **taktiku** na osnovu L1 i CEP činjenica
- L3 pravila (salience 10–20) biraju **konkretni potez** na osnovu izabrane taktike
- Fallback pravila kao poslednje sredstvo ako ni jedno L3 pravilo ne generiše odluku

Konkretni primer rezonovanja

Scenario

Neprijatelj: GoblinMage, nivo 2 (arhitip: `MAGIC_CASTER`)

- HP: 30 / 70 (43%) | Mana: 42 / 45 | Stamina: 3 / 20
- Statistike: napad=4, odbrana=3, magija=9
- Dostupni potezi:
 - `firebolt`: `DAMAGE_MAGIC`, cena: 15 mane
 - `arcaneSurge`: `DAMAGE_MAGIC`, cena: 20 mane
 - `hexShield`: `SELF_BUFF`, cena: 10 mane
 - `manaDrain`: `DRAIN`, cena: 10 stamine → **nije dostupan** (stamina=3 < 10)
- Nema aktivnih statusnih efekata

Igrač: Knight, nivo 2

- HP: 60 / 120 (50%) | Statistike: napad=12, odbrana=10, magija=4
- Nema aktivnih statusnih efekata

Istorija borbe (poslednja 4 događaja na `combat-stream`):

Redosled	Akter	Potez	Kategorija	Šteta
1	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	12
2	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	14
3	Igrač	<code>battleCry</code>	<code>SELF_BUFF</code>	—
4	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	11

Korak 1 — L1 Percepcija (salience 100)

Pravila sa najvišim prioritetom izvlače apstraktne činjenice iz sirovih vrednosti.

EvaluateResourceStaminaStarved:

stamina 3/20 = 15% < 20% → INSERT ResourceStatus (STARVED, "stamina")

EvaluateResourceManaStarved:

mana 42/45 = 93% > 20% → uslov nije ispunjen

DetectStatDisadvantage:

player.attack(12) - enemy.defense(3) = 9 > 5 → INSERT StatComparison (PLAYER_PHYSICAL_DOMINATES)

Template pravilo EvaluateHealthCritical_MAGIC_CASTER:

hpPercent(0.43) ≥ criticalHpPct(0.30)? Da, ali uslov je < 0.30 → nije kritično → PerceivedThreat se ne insertuje

Korak 2 — CEP nad event stream-om

Pravila procesiraju tok borbenih događaja korišćenjem klizajućih prozora.

AssessBurstDamageRisk (accumulate sum, window: poslednje 3 štete):

sum DAMAGE_DEALT od igrača: 12 + 14 + 11 = 37 37 > 70 × 0.35 = 24.5 ✓ → INSERT BurstDamageAssessment (37, HIGH)

DetectPlayerPhysicalSpammer (accumulate count, window: poslednja 4 poteza):

DAMAGE_PHYSICAL potezi igrača: *slash*, *slash*, *battleCry* (ne), *slash* = $3 \geq 3$ ✓ → **INSERT**
PlayerBehaviorProfile (*PHYSICAL_SPAMMER*)

Korak 3 — Backward Chaining: `canKillPlayer`

Pre bilo kakvog taktičkog razmatranja, sistem postavlja cilj: **"Postoji li potez koji može ubiti igrača ovaj potez?"**

Traži dokaz koji zadovoljava cilj: `MoveOption` čija `projectedValue` $\geq$ `player.currentHp`.

Provera dostupnih poteza:

Potez	Projektivna šteta	Player currentHp	Može ubiti?
<code>firebolt</code>	20	60	Ne, $20 < 60$
<code>arcaneSurge</code>	25	60	Ne, $25 < 60$
<code>hexShield</code>	— (nije šteta)	60	Ne

*Query vraća **false** jer ni jedan dostupan potez ne može ubiti igrača → sistem ne emituje kill odluku i nastavlja sa normalnim taktičkim rezonovanjem*

`canKillPlayer` je postavljen eksplicitno, i tek kada se **ne može dokazati**, rezonovanje se nastavlja ka L2 sloju. Da je igrač imao, recimo, 22 HP, `arcaneSurge` bi zadovoljio uslov i rezonovanje bi tu stalo sa odlukom najvišeg prioriteta, bez prolaska kroz L2 i L3.

Korak 4 — L2 Selekcija taktike (salience 35–50)

Pravila taktičkog sloja biraju strategiju na osnovu L1 i CEP činjenica.

`ChooseAggressiveTactic_MAGIC_CASTER` (salience 50):

`hpPercent(0.43) ≥ aggressionHpPct(0.45)?` Ne, $0.43 < 0.45$ → uslov nije ispunjen

`ChooseDebuffPlayerTactic` (salience 50):

`StatComparison(PLAYER_PHYSICAL_DOMINATES)` postoji ✓ `MoveOption(ENEMY_DEBUFF)` dostupan? → `GoblinMage` nema debuff poteze → uslov nije ispunjen

`ChooseConservativeTactic` (salience 45):

```
ResourceStatus(STARVED) postoji ✓ | not Tactic() ✓ → INSERT Tactic(CONSERVATIVE)
```

Korak 5 — L3 Selekcija poteza (salience 10)

`EmitConservativeMove`:

`Tactic(CONSERVATIVE)` ✓ *Bira MoveOption sa najnižom vrednošću costValue među dostupnima:*

Potez	Cena
arcaneSurge	20 mane
firebolt	15 mane
hexShield	10 mane ← minimum

→ **INSERT** `EnemyDecision("hexShield", 6, "Conservative (low-resource) tactic: hexShield")`

Ishod i obrazloženje

GoblinMage odabira `hexShield`, buff-uje se umesto da napada.

Ovo je racionalna odluka koja sledi iz kaskade zaključivanja:

- Stamina mu je kritično niska → `manaDrain` nije dostupan, štednja je poželjna
- Igrač fizički dominira (9 poena razlike) → debuff-ovanje igrača bi bilo idealno, ali potez nije dostupan
- Agresivni prag nije dostignut (HP 43% < 45%) → ofanzivna taktika nije opravdana

Ceo proces, od sirovih statistika do konkretnog poteza, odvija se transparentno kroz čitljiva pravila, sa jasnim tragom zaključivanja koji je moguće ispitati i promeniti u svakom koraku.